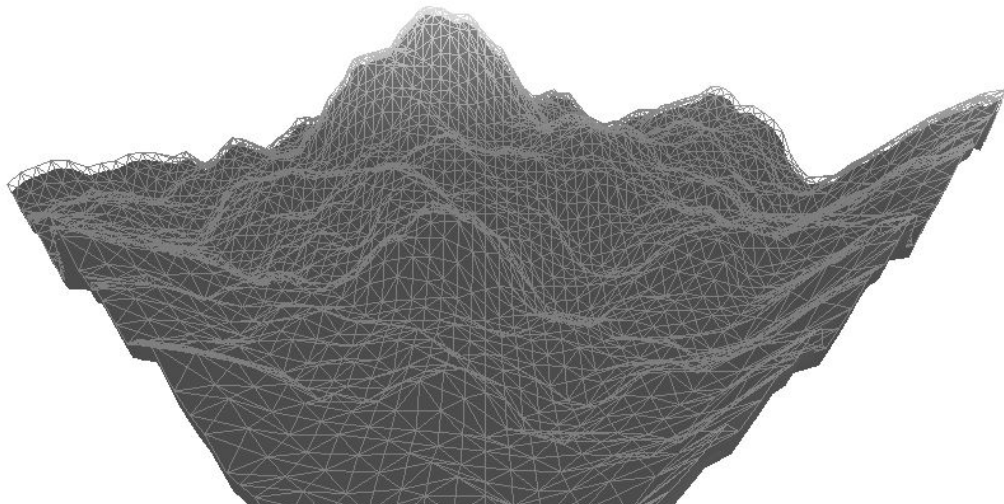


# Pathfinding em Malhas 3D

Análise estatística e comparativa



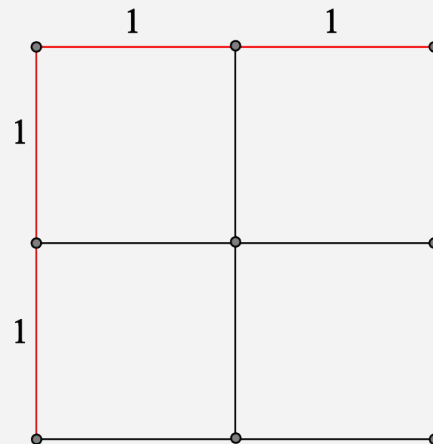
# Introdução e Definição do Problema

Busca de caminhos (ou *pathfinding*): encontrar caminhos entre um ponto de partida e um objetivo por meio da exploração do espaço de busca.

De puramente abstratos para aplicação direta na computação interativa, um componente crucial para o desenvolvimento de jogos e simulações em tempo real.

# Introdução e Definição do Problema

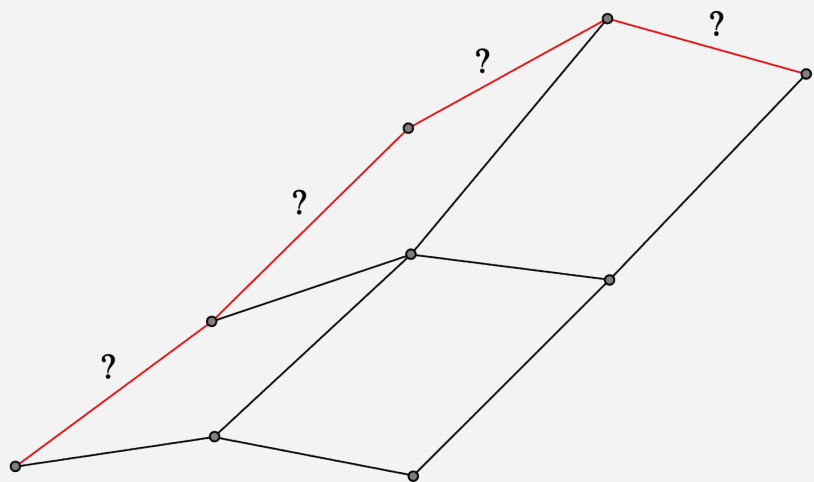
Em espaços 2D, a passagem entre coordenadas **inteiras** é uniforme e fácil de se navegar.



# Introdução e Definição do Problema

No espaço 3D, é preciso considerar a variação de altura, inclinação do relevo e obstáculos complexos

Isso traz instabilidade e inconsistências matemáticas nos cálculos de distância



## Objetivo Geral

Analisar comparativamente e estatisticamente o desempenho de algoritmos de busca altamente usado em múltiplos cenários 3D.

## Objetivos Específicos

1. Implementar uma biblioteca de grafos eficiente para execução de algoritmos de pathfinding.
2. Desenvolver uma infraestrutura de renderização para visualização das malhas e caminhos encontrados.
3. Adaptar a arquitetura para execução concorrente e testes de estresse.
4. Desenvolver geradores procedurais de malhas para diferentes cenários de teste.
5. Analisar estatisticamente os resultados obtidos nos testes.

# Justificativa

- Performance em tempo real
- Lacuna Teórica em 3D
- Controle Metodológico

Existe uma grande lacuna em estudos de busca de caminhos na área de renderização 3D.

Pesquisas recentes tendem a focar unicamente em ambientes 2D ou em um único modelo de interpretação de malhas.

Este estudo visa superar essa barreira e comparar diferentes estilos de modelagem de terreno.

# Justificativa

- Performance em tempo real
- Lacuna Teórica em 3D
- Controle Metodológico

Adicionalmente, existe a necessidade de separar a análise comparativa dos grandes motores gráficos (Unity, Unreal e outras) e trazer uma visão única e sem ruídos em uma engine criada do zero.

Isso nos permite medir a performance de CPU e memória de forma pura.



# Trabalhos Relacionados e Diferencial Técnico

| Critérios         | Proposta                                   | Johansson (2024)   | Kapi (2022)      |
|-------------------|--------------------------------------------|--------------------|------------------|
| Dimensão Espacial | Malhas 3D volumétricas, Octotrees e Voxels | Voxels (Blocos 3D) | Grades 2D planas |
| Geometria         | Procedural                                 | Cubos fixos        | Matrizes planas  |
| Algoritmos        | Dijkstra, A*, JPS e Theta*                 | A* e Dijkstra      | A*, JPS e Theta* |
| Arquitetura       | Paralela                                   | Sequencial         | Sequencial       |
| Ambiente/Carga    | Renderização 3D concorrente                | Execução isolada   | Execução isolada |

---

# Referencial Teórico e Metodologia

---

# Grafos

$$G = (V, A)$$

Onde:

- $G$  é um grafo
- $V$  é um conjunto de vértices
- $A$  é um conjunto de arestas

Vértices são entidades individuais que representa algo, no nosso caso coordenada. Arestas representam a travessia de um vértice para outro e carrega consigo o peso dessa transição.

Um algoritmo de busca é o procedimento que atravessa os vértices do grafo a partir das arestas e, com seus respectivos pesos, encontra o caminho de menor custo entre dois vértices.

# Grafos

- Dijkstra: busca exaustiva.
- A\*: Uso de heurísticas espaciais.
- JPS: Otimização para “saltar” blocos vazios.
- Theta\*: Busca em qualquer ângulo, suaviza o caminho.

O algoritmo de Dijkstra é o algoritmo fundamental de caminho mínimo e o “pai” dos principais algoritmos da atualidade. Com a adição de heurísticas ao seu processo, outros algoritmos como A\*, JPS e Theta\* foram desenvolvidos.

Enquanto o Dijkstra verifica apenas o peso real das aresta, os demais se baseiam na soma do peso e o valor heurístico  $f(n) = g(n) + h(n)$ .

# Grafos

Heurísticas espaciais:

- Manhattan:  $h(n) = |x_1 - x_2| + |y_1 - y_2|$
- Euclidiana:  $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
- Diagonal:  $h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$

Uma malha 3D (ou poligonal) é a forma de um objeto em um espaço tridimensional. Elas são compostas por:

- Vértices: Os pontos no espaço 3D, definidos por  $(x, y, z)$ .
- Arestas: As linhas retas que conectam dois vértices.
- Faces: As superfícies preenchidas delimitadas por arestas.

Já evidenciando sua possível relação com grafos...

# IFCG

$(0.5, 1, 0.5)$



$(1, 0, 1)$



$(1, 0, 0)$



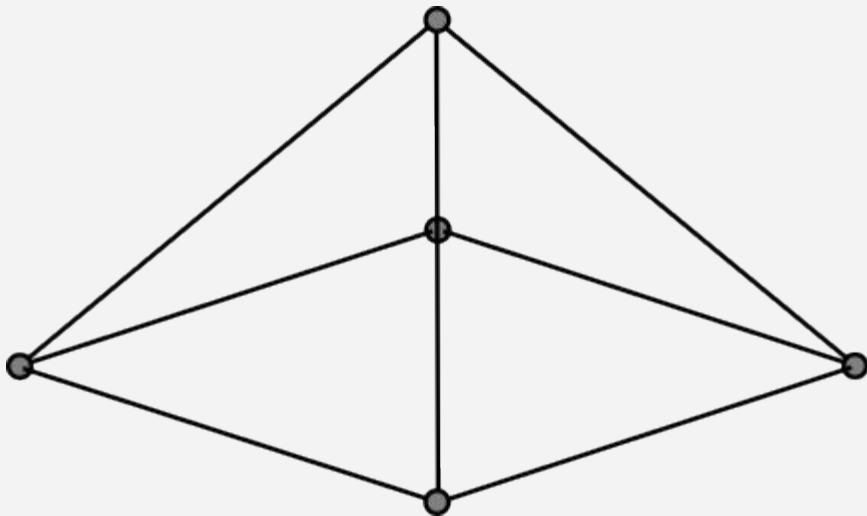
$(0, 0, 1)$



$(0, 0, 0)$

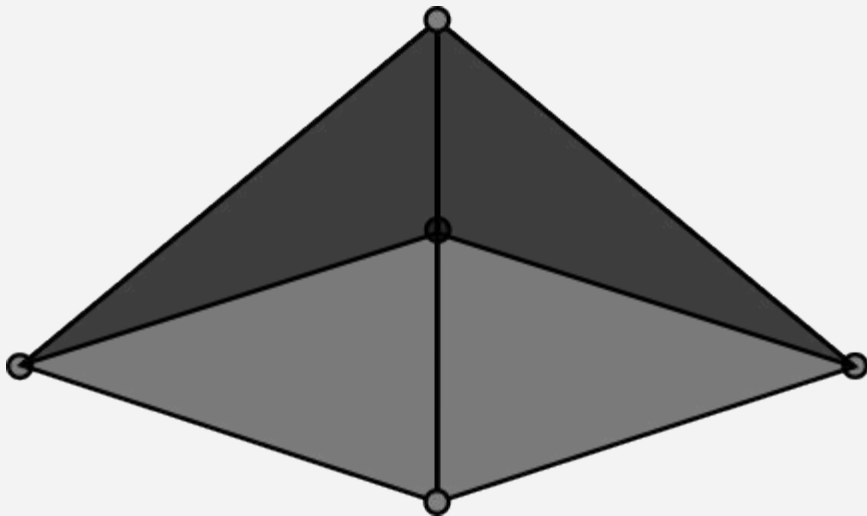


IFCG

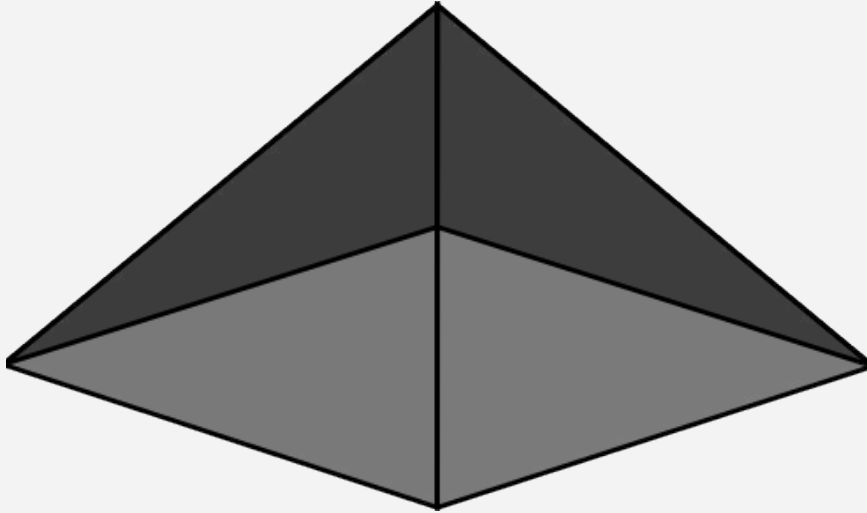




IFCG



IFCG



# Geração Procedural

Para garantir uma grande quantidade de cenários de teste, foi utilizado Ruído de Perlin para gerar de forma automatizada as malhas de teste.

O algoritmo funciona empilhando múltiplas camadas ligeiramente diferentes (oitavas) de ruído para formar uma imagem contínua e suave.

# Geração Procedural

Em uma oitava, o espaço da imagem é dividido em uma grade e em cada intersecção é atribuído um vetor de gradiente aleatório. Para cada pixel da imagem, é calculado o vetor que vai dos nós da grade até esse ponto e é feita uma multiplicação escalar. Os valores calculados nos nós vizinhos são interpolados e finalmente um valor é atribuído ao pixel.

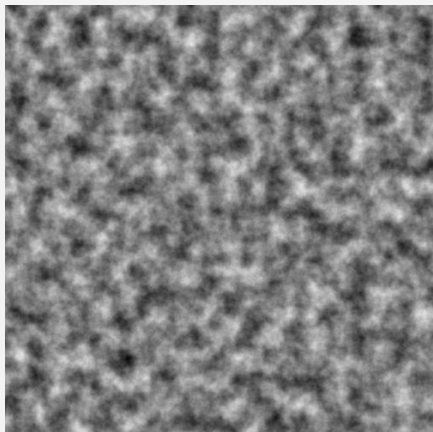
# Geração Procedural

Para que as oitavas não sejam idênticas, valores de lacunaridade (frequência) e persistência (amplitude) influenciam no cálculo dos valores finais dos pixels.

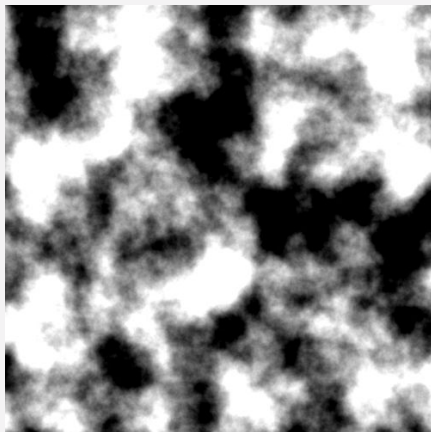
A frequência define a quantidade de detalhes , enquanto a amplitude define a escala de intensidade desses detalhes.

# Geração Procedural

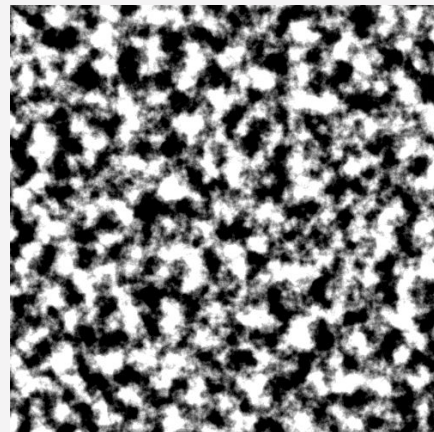
Ruído de alta  
frequência



Ruído de alta  
amplitude



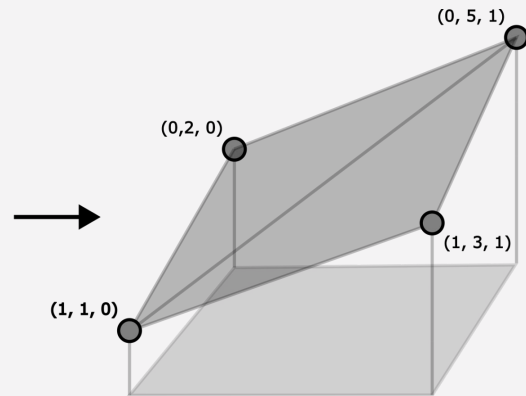
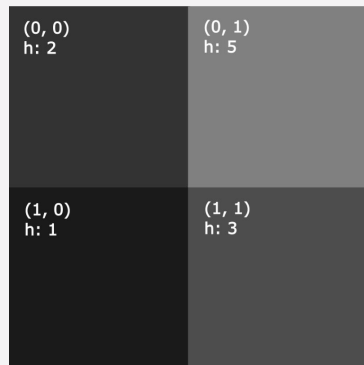
Ruído de alta  
frequência e  
amplitude



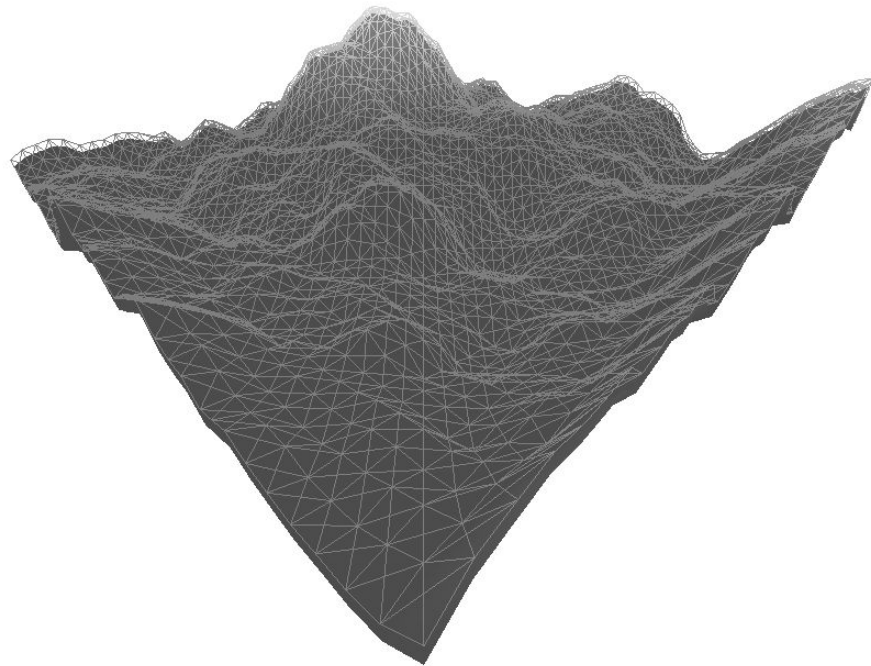
# Geração Procedural

Com uma imagem de ruído gerada, é possível traduzir seus valores para uma malha e grafo.

Convertendo a posição dos seu pixels para as coordenadas x e z dos vértices e convertendo seus valores de cor para a altura.



# Geração Procedural







Todo o ambiente de teste e renderização será paralelizado pela classe TaskMaster.

Essa classe encapsula conceitos de Monitor, Task Scheduler e Filas Multinível para garantir um ambiente concorrente limpo e seguro. Também, permitindo executar testes enquanto o motor gráfico desenvolvido roda no fundo.

**Paralelização**

## Desenho Experimental

Os algoritmos serão testados em diferentes cenários de teste, alternando entre possíveis configurações de ruídos:

- Escala: Tamanho da malha ( $N \times N$ ).
- Lacunaridade: Quantidade de obstáculos.
- Persistência: Nível de dificuldade dos obstáculos.

## Desenho Experimental

Em todos os testes serão testados:

- Tempo de CPU
- Consumo de memória
- Qualidade do caminho (custo)
- Taxa de erro de cache

Será priorizado um ambiente quiescente para não haver interferências do sistema operacional.

# Riscos Associados ao Projeto

| Risco                                                              | Mitigação                                                                          |
|--------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Complexidade de adaptar algoritmos complexos como JPS 3D e Theta*. | Referências de alto valor, uso de bibliotecas de teste e otimização de compilação. |
| Coleta de amostra e viés de dados.                                 | Controle rígido do ambiente de teste e forte embasamento teórico.                  |
| Expansão ambiciosa e insuficiência do tempo planejado.             | Cronograma modular, priorização de adições e ambiente de pesquisa estável.         |

# Desenvolvimento e Resultados Parciais

# Resultados esperados

- Distinguir melhores algoritmos para cada modelo geométrico e tipo de cenário.
  - Dijkstra com pior tempo de execução.
  - JPS com melhor tempo em áreas planas.
  - Theta\* com melhor qualidade de caminhos.
- Validar a influência da escala e dificuldade de terreno no tempo de execução de algoritmos de busca.
  - ANOVA com  $p\text{-valor} < 0,05$

# Cronograma de Atividades

## Primeiro semestre

| <b>Etapas</b>                             | <b>Período</b> | <b>Status</b> |
|-------------------------------------------|----------------|---------------|
| Revisão Bibliográfica                     | Jan-Fev/2026   | Concluído     |
| Motor Gráfico e Grafos                    | Fev-Abr/2026   | Concluído     |
| Algoritmos de Busca                       | Mar-Mai/2026   | Concluído     |
| Geração de Cenários                       | Abr-Mai/2026   | Concluído     |
| Concorrência                              | Mai-Jun/2026   | Concluído     |
| Planejamento da coleta de dados e análise | Mai-Jun/2026   | Concluído     |
| Defesa do Pré-Projeto                     | Jun/2026       | Concluído     |

# Cronograma de Atividades

## Segundo semestre

| <b>Etapas</b>                 | <b>Período</b> | <b>Status</b> |
|-------------------------------|----------------|---------------|
| NavMeshes, Octotrees e Voxels | Jul-Set/2026   | Pendente      |
| Testes de Estresse            | Ago/2026       | Concluído     |
| Algoritmos Adicionais         | Ago-Set/2026   | Pendente      |
| Coleta Automatizada de Dados  | Set-Out/2026   | Pendente      |
| Análise estatística           | Out-Nov/2026   | Pendente      |
| Redação Final                 | Out-Dez/2026   | Pendente      |
| Defesa do TCC                 | Dez/2026       | Pendente      |



# Considerações Finais e Conclusão